Computers & Security 138 (2024) 103665

Contents lists available at ScienceDirect

Computers & Security

journal homepage: www.elsevier.com/locate/cose

Unveiling shadows: A comprehensive framework for insider threat detection based on statistical and sequential analysis

Haitao Xiao [a,b], Yan Zhu [a,b], Bin Zhang [c], Zhigang Lu [a,b], Dan Du [a,b,*], Yuling Liu [a,b]

[a] Institute of Information Engineering, Chinese Academy of Sciences, Beijing, China
[b] School of Cyber Security, University of Chinese Academy of Sciences, Beijing, China
[c] China Cybersecurity Review Technology and Certification Center, Beijing, China

ARTICLE INFO

Keywords:
Insider threat detection
Statistical analysis
Sequential analysis
Deep learning

ABSTRACT

With the increasing importance of internal information security, detecting insider threats has become a critical issue to safeguard organizations' information systems. However, most of the previous studies either overlook temporal relationships or have difficulty attaining accurate performance. One of the primary factors contributing to this challenge is their approach, which lacks a holistic perspective. To our knowledge, none of these studies has considered the integration of statistical and sequential information in addressing this issue. Therefore, we propose a comprehensive framework for insider threat detection based on statistical and sequential analysis to address this challenge. Leveraging the strengths of both statistical analysis and sequential analysis, we deploy an efficient implementation for analyzing and modeling user data based on convolutional attention and a transformer encoder, referred to as CATE. First, user behavior logs are consolidated from diverse sources and preprocessed into a suitable format for subsequent analysis. Then, two parallel analysis modules analyze user data in two different dimensions. The analysis modules are entirely constructed using a neural network for its high adaptability and efficient integration of information from distinct dimensions. Specifically, a subnetwork structure based on convolutional attention is designed to effectively learn statistical information, while a separate subnetwork structure based on transformers is tailored for learning sequential information. Finally, we perform a series of solid experiments utilizing the publicly available CERT dataset to evaluate our framework's effectiveness and robustness in detecting insider threats and identifying malicious scenarios.

1. Introduction

One of the most significant digital threats to an organization's information system infrastructure is insider threats, which refer to harmful threats from people inside an organization or enterprise. Detecting insider threats is particularly challenging compared to external threats due to the insiders' access privileges within the organization's information systems, coupled with their familiarity with the organizational structure. Additionally, the elusive and constantly evolving characteristics of insider threats contribute to the complexity of detection efforts (Yuan and Wu, 2021).

In the contemporary landscape, the cyber threat posed by malicious insiders is on the rise. Based on findings from the Global Report on Insider Threats' Cost for the year 2022, as indicated by Ponemon (2022), incidents related to insider threats have surged by 44% within the past two years. Moreover, the associated costs per incident have risen by over a third, now averaging $15.38 million. The 2023 Insider Threat Report, released by Gurucul (2023), underscores that 74% of organizations affirm the increasing frequency of insider attacks. Over half of organizations have encountered an insider threat within the past year, with 8% facing such threats on more than 20 occasions. Since insider threats are becoming more serious and frequent, it is essential to identify them accurately and promptly.

To tackle the intricate and steadily growing challenge of detecting insider threats, current research mainly employs machine learning or deep learning techniques due to their innate learning capacity. Previous studies, distinguished by their diverse feature extraction techniques, can be generally categorized into two primary domains: statistical-based studies and sequential-based studies. Statistical-based studies commonly utilize mathematical and statistical techniques to

* Corresponding author at: Institute of Information Engineering, Chinese Academy of Sciences, Beijing, China.
E-mail addresses: xiaohaitao@iie.ac.cn (H. Xiao), zhuyan0117@iie.ac.cn (Y. Zhu), zhangbin@isccc.gov.cn (B. Zhang), luzhigang@iie.ac.cn (Z. Lu), dudan@iie.ac.cn (D. Du), liuyuling@iie.ac.cn (Y. Liu).

https://doi.org/10.1016/j.cose.2023.103665
Received 28 September 2023; Received in revised form 29 November 2023; Accepted 14 December 2023
Available online 19 December 2023

extract features from user behavior logs. Subsequently, these features are employed to train machine learning or deep learning models with the purpose of detection. Sequential-based studies, on the other hand, leverage the temporal ordering of log events to model patterns in user behavior. These patterns are then applied for the detection of anomalous user behavior.

While these preceding investigations have demonstrated the ability to attain relatively robust performance, they are beset by various limitations: (1) Statistical-based studies provide simplicity and efficiency in analyzing static data with distinct features, but they frequently overlook temporal relationships and encounter challenges in adapting to dynamic threats. (2) Sequential-based studies effectively capture temporal dependencies and are capable of handling dynamic threats. Nonetheless, they require high-quality user sequence data, and their performance is compromised by the heterogeneous nature of user data. (3) Existing studies do not integrate statistical and sequential information for insider threat detection. Consequently, researchers in either paradigm forfeit a portion of valuable information, which leads to inefficient utilization of the raw user behavior logs.

To overcome the constraints inherent in prior research on the identification of insider threats, we propose a comprehensive insider threat detection framework based on statistical and sequential analysis and deploy an efficient implementation using **C**onvolutional **A**ttention and **T**ransformer **E**ncoder (CATE). Our framework aims to effectively improve the detection rate and reduce the false-positive rate in insider threat detection by seamlessly integrating statistical and sequential information from user behaviors. Initially, to make efficient utilization of the raw user behavior logs, we consolidate log files from diverse sources and harmonize them into the required format for subsequent phases of behavior analysis. Second, for enhanced learning from both statistical and sequential information, we devise two analysis modules based on improved convolutional neural networks and transformers. These modules comprehensively extract patterns from two distinct dimensions of information. Last, the classification module's objective is the effective categorization of patterns extracted from the aforementioned analysis modules. This process facilitates the identification of potential insider threat behaviors. The principal contributions of this paper are as follows:

- We propose a comprehensive insider threat detection framework that utilizes both statistical and sequential analysis and deploys an efficient implementation for analyzing and modeling user data based on convolutional attention and a transformer encoder called CATE. It integrates both statistical and sequential information, providing a comprehensive representation of user behavior from two distinct perspectives. The integration of statistical and sequential information has not been employed in previous studies.
- We develop straightforward statistical feature sets and a streamlined behavior sequence encoding strategy. They effectively capture user behavior patterns from two distinct perspectives within user behavior logs, demonstrating their advantage in enhancing the performance of our presented framework for detecting insider threats.
- Two subnetwork modules in the same neural network are presented in this framework. In the design of the statistical analysis module, we incorporate a convolutional neural network and self-attention mechanism. This combination empowers the model to focus on essential features and enhance their representation, thus providing a better expression of users' statistical behavior patterns. In the design of the sequential analysis module, we introduce the transformer model to learn from the encoded user behavior sequences, effectively capturing the temporal relationships among user behaviors.
- Solid experiments are conducted using the public CERT dataset to assess the performance of our framework. Comprehensive experiments show the effectiveness of CATE in comparison to other existing methods. CATE exhibits strong competitiveness in precisely detecting insider threats and identifying malicious scenarios. Additionally, we conduct an in-depth analysis of the limitations of the CATE method in identifying malicious scenarios, providing substantial insights for future research.

The remaining sections of this paper are structured as follows: Section 2 conducts a review of prior research regarding insider threat detection. Section 3 introduces our framework and provides details of its implementation. Following that, in Section 4, we outline the experimental configuration and show the experimental results in the next section. Finally, we conclude this paper by summarizing our proposed framework's details and discussing potential directions for future research.

2. Related work

Insider threats have become a pressing issue for governments and organizations, garnering significant attention from researchers worldwide. Numerous organizations are confronted with the peril of insider threats capable of inflicting substantial harm to both their assets and reputation. Differentiating between legitimate users and malicious insiders can be a challenging task because insiders often possess familiarity with the organization's internal structure and have access to its internal information systems. To address this challenge, researchers have developed multiple approaches to mitigate insider threats. Depending on distinct feature extraction techniques, existing approaches can be categorized into two primary groups: statistical-based insider threat detection approaches and sequential-based insider threat detection approaches.

2.1. Statistical-based insider threat detection approaches

Researchers extract statistical features from logs of user behavior in statistical-based insider threat detection approaches. These features encompass activities such as user logins, usage of removable devices, file access, email communication, web browsing history, and other related behavioral data. Subsequently, machine learning and deep learning algorithms are commonly employed to train models aimed at detecting insider threats.

In the early stages of statistical-based approaches to insider threat detection, a strategy emerged that involved the analysis of user behavior, followed by the formulation of corresponding rules. For example, Nguyen et al. (2003) formulated a set of regulations to identify uncommon file system-related system calls, leading to the efficient detection of established abnormal behaviors. The model presented in the study by Li and Liu (2020) placed emphasis on the development of rules and regulations designed to address and identify breaches effectively. Nevertheless, this rule-based approach heavily relied on domain expertise and lacked the capacity to address insider threats that had not been encountered before.

The rising popularity of artificial intelligence has prompted a significant number of researchers to delve into machine learning and deep learning methods for identifying insider threats. Le et al. (2020) introduced a system for detecting insider threats, employing machine learning techniques. They extracted a collection of information-rich features encompassing various details, including computer usage, timing of actions, and other specific action-related characteristics. Then, they employed algorithms such as logistic regression, random forest, neural network, and XGBoost to model extracted features, enabling the prediction of insider threats. Bartoszewski et al. (2021) proposed an objective evaluation and comparative analysis of different machine learning models for detecting insider threats, encompassing local outlier factor (LOF) and ensemble methods.

An approach employing an autoencoder for detecting insider threats was introduced by Liu et al. (2018). They employed a deep autoencoder

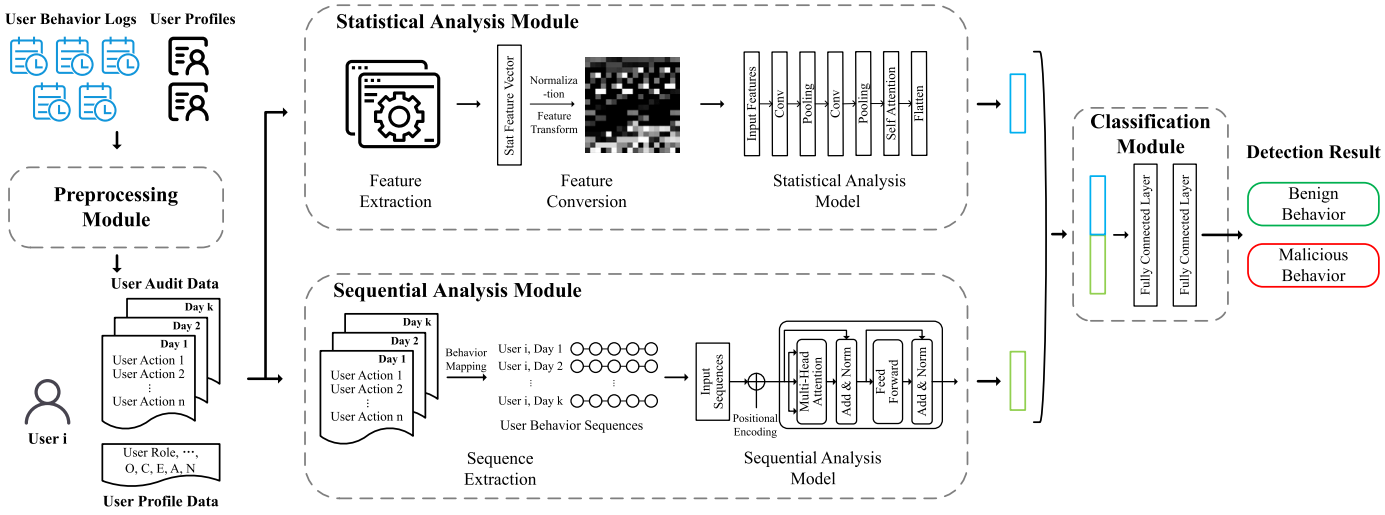


Fig. 1. Overview of the proposed framework.

to learn a low-dimensional representation of normal behavior from a specific category of audit data on user activities. Subsequently, they utilized the autoencoder-generated reconstruction error as a metric for detecting deviations in user behavior. Zhang et al. (2020) employed a denoising autoencoder for the detection of anomalous data. The model employed integrated techniques such as the Gaussian mixture model and OCSVM to detect anomalous data. Gayathri et al. (2020) derived features from the usage patterns of insiders and then transformed them into image representations. Subsequently, they leveraged pretrained deep convolutional neural networks for the purpose of anomaly detection, aiming to detect malicious insiders. Li et al. (2021a) also utilized user behavior logs to extract behavioral patterns, which were achieved by converting these logs into image-based representations. Additionally, they applied geometric transformations to these images, resulting in the creation of a more extensive training dataset.

Despite the efficacy of statistical-based approaches in identifying behavioral features, they frequently overlook the temporal relationships among user behaviors. Consequently, the statistical-based approach is vulnerable to failing in the detection of insider threat attacks that possess the ability to evade legitimate features and struggle to adapt to dynamic threats.

2.2. Sequential-based insider threat detection approaches

Sequential-based approaches analyze log data as a sequence of events to identify behavioral patterns indicative of insider threats.

Researchers typically convert behavioral logs into sequential data, subsequently employing sequence models for the analysis of user behavior. Du et al. (2017) presented an anomaly detection method known as DeepLog, which was built upon the foundation of long short-term memory. DeepLog was crafted to autonomously acquire log patterns during standard executions and recognize deviations as anomalies when observed log patterns diverge from the model established through log data from regular operations. An insider threat detection approach employing a deep neural network was introduced by Yuan et al. (2018). They employed LSTM to capture temporal relationships within the sequence and used a CNN for classification. Wang et al. (2019a) proposed an embedding learning approach using heterogeneous event sequences for insider threat detection. They modeled user activities and their relationships using a heterogeneous event sequence derived from log data. Subsequently, they applied an embedding learning technique to acquire a low-dimensional vector representation for each user, utilizing the event sequence as the basis for learning. He et al. (2021) proposed an approach for insider threat detection. This approach relied on historical user behavior data and incorporated attention mechanisms. They used an LSTM network to learn user behavior sequences and integrated an attention mechanism to capture individual user behavior patterns. Huang et al. (2021) introduced a framework for insider threat detection that leveraged temporal-semantic representations of user behavior. They infused temporal information into behavior and utilized a language model that had been pretrained to capture the fused semantic representation. From the audit log files, Zhu et al. (2022) extracted sequences depicting patterns of user resource access and then proceeded to implement data augmentation specifically on minority class sequences. This approach was undertaken to address the issue of data imbalance.

Although the sequential-based approach has the advantage of capturing the correlation and temporal relationship among user behaviors, it is limited by the amount of information that can be conveyed in the sequence. Consequently, the approach may not capture the full complexity of the user's behavior in detail, necessitating further investigation and refinement of the methodology.

By summarizing the deficiencies in existing studies, we introduce a novel and comprehensive insider threat detection framework that utilizes both statistical and sequential analysis. Our proposed framework achieves complementary strengths by combining information from two distinct perspectives. We design two separate analysis modules for learning statistical and sequential information. The combination of the two analysis modules enables comprehensive analysis of user behavior logs and makes insider threat detection more effective. The following section delves into the comprehensive design and implementation of the proposed framework.

3. Design and implementation

In this section, we initially outline the workflow of our proposed framework and then provide an overview of the implementation to illustrate the functionality of each module. Then, we elaborate on the implementation details of each module, demonstrating how these modules analyze internal user behavior from both statistical and sequential perspectives.

3.1. Framework and module overview

The architecture of our proposed framework, comprising three distinct steps and four core modules, is depicted in Fig. 1. The first step involves merging and preprocessing log files from various sources in chronological order to obtain the required format for subsequent behavior analysis phases. The second step encompasses two parallel processing modules dedicated to analyzing user behaviors from two perspectives. The two processing modules effectively characterize user behavior

Table 1
Statistical feature structure.

Data Source	Type	Activity	PC	Time
User Audit Data	Logon	Logon/Logoff times	Own PC,	WorkTime,
	Device	Connect/Disconnect times, File Tree length	Shared PC, Supervisor's PC,	OffworkTime, Weekday,
	File	Open/Write times, File depth, type, Content length, words	Other PC	Weekend
	Email	Send/Receive times		
	HTTP	Web View times, URL length, depth, HTTP type		
User Profile Data	Role, Project, Unit, Dept, Team, Supervisor, OCEAN			

from the perspectives of statistical and sequential information. The third and final stage involves the classification module, which is tasked with efficiently categorizing the patterns extracted from the preceding two analysis modules. This classification process is aimed at identifying potential insider threat behaviors. Our framework can provide valuable insights for insider threat detection and prevention.

3.2. Preprocessing module

The preprocessing module undertakes multisource log integration and time granularity division. Due to variations in the values and meanings of data from different sources, direct utilization of the original log data by subsequent analysis modules becomes challenging. Hence, the preprocessing module plays a crucial role in the merging of original multisource logs into the specified format required by the subsequent modules within the framework. Specifically, the module comprises two successive procedures as follows:

1. Multisource log integration: User behavior logs from various sources are first integrated into a unified primary data table (User Audit Data), organized by user ID, with each action chronologically ordered. A distinct primary data table corresponds to each user. Second, organizational structure information and psychometric data (User Profile Data) for each user are consolidated into a table capturing the user's roles within the organization, departmental affiliations, and psychometric assessments encompass a range of scores, including measures of openness, conscientiousness, extraversion, agreeableness, and neuroticism (Cullen et al., 2011). These consolidated data are then directly fed into the statistical analysis module to construct statistical features.
2. Time granularity division: For integrated user primary data tables, the time granularity division will divide the user primary data tables into daily segments. Each file represents all the user actions conducted by a user throughout that specific day. The user actions within the divided file remain arranged chronologically. If there are malicious activities in the user behavior actions for a day, that user day will be labeled malicious (1). Conversely, if no such activities are present, the user day is categorized as benign (0). The above preprocessing of user behavior logs will be directly supplied to the subsequent modules for statistical analysis and sequential analysis.

By performing these essential tasks, the preprocessing module ensures seamless compatibility between the heterogeneous log formats and the requirements of subsequent modules, thereby facilitating effective data analysis and processing.

3.3. Statistical analysis module

As a core element of the analysis framework, the statistical analysis module performs thorough statistical analyses of both user audit data and user profile data. Its primary objective is to characterize the daily behavior patterns exhibited by internal users, along with the structure and psychometric information of individuals within the organization. Through the analysis of statistical information, this module enhances the overall comprehension of internal user behavior, facilitating the effective detection and analysis of potential insider threats.

Previous studies have developed numerous features for statistical analysis. Nevertheless, the construction process for these features has been frequently burdensome and intricate. To address this issue, our proposed statistical analysis module simplifies the extracted features and introduces a convolutional neural network and attention-based models to efficiently model our extracted features. Our objective is to refine and improve the effectiveness of extracting features, streamlining the feature extraction process. The extracted features are normalized and subsequently converted into a two-dimensional matrix format. Finally, we employ an adapted convolutional neural network model to analyze the statistical features encapsulated within this two-dimensional matrix format, generating an ultimate statistical analysis vector as the resultant output. Through our statistical analysis module, a comprehensive statistical analysis of user behavior patterns is facilitated.

In the following paragraphs, we provide detailed implementation specifics for the statistical analysis module within this framework. These details are presented across three distinct steps: feature extraction, feature conversion, and statistical analysis model construction.

3.3.1. Feature extraction

Within the statistical analysis module, the process of feature extraction plays a pivotal role, as the performance of this module relies on the quality of the extracted features. We design a simplified and reasonable feature set derived from two distinct data sources, namely user audit data and user profile data, with a focus on identifying potential malicious scenarios. These carefully designed features aim to empower statistical analysis models to effectively analyze daily user behavior patterns. By incorporating these characteristic features, our module enhances the capability of statistical analysis models to detect and analyze anomalous behavior, contributing to the overall effectiveness of our insider threat detection framework.

User audit data serve as a valuable information source for the statistical analysis module, encompassing records of user logins, file access, device connections, email communications, and web browsing. These data sources form a robust foundation for conducting statistical analyses of user behavior. Furthermore, user profile data complement the statistical analysis module by providing essential background information about the users. This data encompasses personal attributes, organizational roles, and interpersonal relationships, among other relevant attributes. Through the integration of user profile data, the statistical analysis module acquires supplementary information that facilitates a more comprehensive analysis of users' statistical behavior. The integration of user audit data and user profile data enables the statistical analysis module to generate insightful patterns and meaningful statistics, contributing to a comprehensive understanding of internal user behavior and enhancing the detection rate of insider threat detection.

The feature structure extracted by the proposed statistical analysis module is shown in Table 1. Leveraging the two categories of user data supplied by the preprocessing module, the feature extraction process focuses on extracting features from the user audit data. It is noteworthy that malicious behavior among internal users frequently emerges within the context of their routine work activities. Therefore, statistical features are extracted at distinct levels: activity, PC, and time.

For instance, focusing on the activity level, we incorporate a range of indicators tailored to different types of audit logs, effectively capturing and characterizing the behavioral patterns demonstrated by users. These indicators encompass factors such as the frequency of logon and logoff times in the login type, file open and write occurrences, and diverse file type usage. The indicators utilized in our analysis can be categorized into two distinct types. The first consists of frequency-based indicators, which tally the occurrences of specific behaviors within a predefined timeframe. The second comprises statistics-based indicators, which capture statistical values associated with the observed behaviors. These statistical metrics encompass measures such as the mean, standard deviation, and others, enabling a comprehensive characterization of user behavior patterns. On the level of PC, the process of feature extraction involves categorizing PCs into four distinct types: the user's own PC, shared PC, supervisor's PC, and other PC. This categorization aids in characterizing the user's behavioral patterns across various PC types. Additionally, the time component is considered across four cases: working hours, nonworking hours, weekdays, and weekends. The time dimension provides valuable insights into user behavior during specific time periods.

In summary, the feature vector derived from the audit data encompasses the three aforementioned dimensions, namely activity, PC, and time. It serves as a comprehensive enumeration of the diverse components detailed in Table 1. Moreover, the user profile data are seamlessly integrated into the feature vector, providing vital background information. This integration ensures a comprehensive representation of user statistical behavior within the proposed module.

3.3.2. Feature conversion

The feature vectors extracted above cannot be directly utilized within the statistical analysis model. Thus, a transformation step is necessary. First, we will subject the extracted feature vectors to min-max normalization, constraining their value range to the interval [0, 1]. This normalization procedure serves to mitigate disparities in feature magnitudes, averting any adverse effects on the analysis model. Subsequently, the normalized feature vectors are converted into 18×18 feature matrices. These matrices serve as the input for the statistical analysis model, enabling a structured and comprehensive analysis of the transformed features.

3.3.3. Statistical analysis model construction

In our statistical analysis model, we incorporate a convolutional neural network (Gu et al., 2018) with an attention-based architecture, as depicted in Fig. 2. The convolutional neural network, known for its effectiveness in tasks such as image recognition (Liu et al., 2017), object detection (Xie et al., 2021), document classification (Afzal et al., 2015), and image classification (Sun et al., 2020), exhibits remarkable capabilities in efficiently recognizing local features. By employing convolution and pooling operations, the convolutional neural network effectively handles spatial relationships, enabling the model to comprehend spatial structures within matrices. Consequently, this facilitates the extraction of relevant and informative features. Furthermore, we have introduced the self-attention mechanism, as outlined in Algorithm 1. This mechanism empowers the model to autonomously identify pivotal regions or features within the matrix, thus allocating increased attention to those key components. As a result, the model enhances its perception of important features, thereby improving its representational capacity concerning key attributes. The attention mechanism contributes to enhanced feature representation by dynamically assigning weights to augment or suppress features across various locations. By precisely capturing significant features in different matrix regions and their interdependencies, the model demonstrates improved performance in analyzing the statistical behavior patterns exhibited by users.

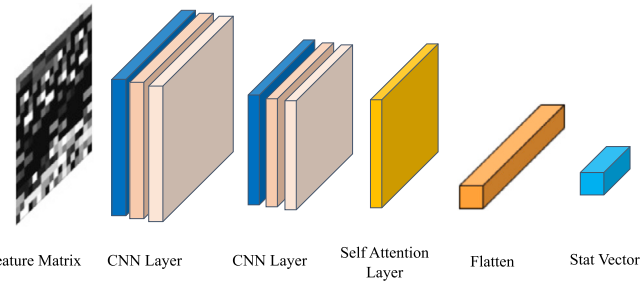


Fig. 2. Statistical analysis model.

Algorithm 1 Self attention.

Require: Input data X (input feature matrix)
Ensure: Output data Y (feature matrix after self-attention)
Initialize W_{query} (weight matrix), W_{key} (weight matrix), W_{value} (weight matrix)
Compute Query, Key, and Value
$Q = X \cdot W_{query}$
$K = X \cdot W_{key}$
$V = X \cdot W_{value}$
Compute attention weights
$attention\_weights = softmax\left(\frac{Q \cdot K^T}{\sqrt{d_k}}\right)$
Compute weighted sum
$weighted\_sum = attention\_weights \cdot V$
Output $Y = weighted\_sum$

The implementation details for the statistical analysis model are described in the following paragraphs.

First, the transformed feature matrix undergoes processing through a convolutional neural network, which consists of a convolutional layer followed by a pooling layer. The input is a single-channel image, which undergoes sequential convolutional and pooling operations to extract behavioral patterns from the feature matrix. After each convolutional layer, there is a subsequent application of batch normalization and a ReLU activation function. The convolution operation utilizes distinct kernels to extract a variety of matrix features, concurrent with the pooling operation, which diminishes the feature map's dimensions.

Next, the feature map from the convolutional neural network is fed into the self-attention layer, aimed at capturing significant information within the feature map. This attention layer encompasses three linear transformations: query Q, key K, and value V. The corresponding Q, K, and V are generated by applying three distinct matrix multiplications to the input feature map. Following this, attention weights are computed using Q and K through dot-product attention, and then these weights are used to linearly combine V, resulting in a weighted sum of the feature maps. Finally, the attention layer output is acquired by scaling it with a learnable factor and adding it to the input feature map.

Finally, the self-attention mechanism's output is directed into a flatten layer for further processing. This layer assists in propagating the feature map after convolution and the self-attention layer, ultimately mapping it to the final output space. The flatten layer takes the feature maps from the self-attention layer and reshapes them into a one-dimensional vector. This flattened feature vector retains both spatial information and high-level features from the original statistical feature matrix.

3.4. Sequential analysis module

As another core component of the analysis framework, the sequential analysis module assumes a vital role in extracting user behavior sequences and formulating a model for analyzing these sequences. First,

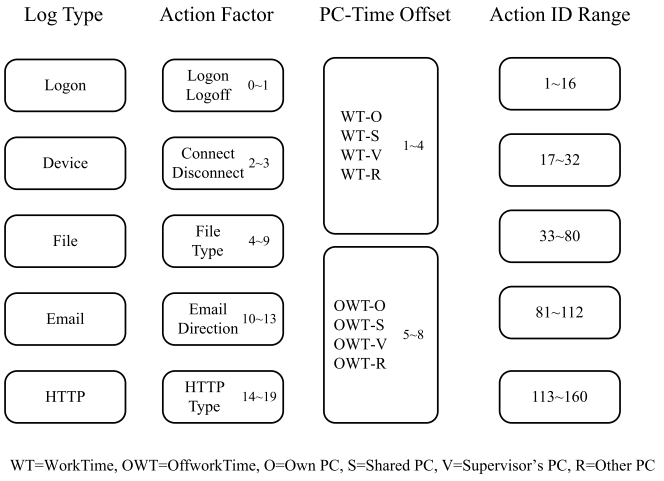


Fig. 3. Behavior mapping strategy.

the objective of the user behavior sequence extraction component is to convert heterogeneous user behavior logs into serialized data that encompass temporal information, enabling a comprehensive characterization of user sequential information. Next, to effectively analyze the extracted user behavior sequences, the sequential analysis model employs a neural network model based on the transformer encoder architecture. The forthcoming paragraphs delve into the implementation details of the sequential analysis module, focusing on two pivotal stages: sequence extraction and sequential analysis model construction.

3.4.1. Sequence extraction

During the sequence extraction phase, we retrieve user behavior sequences from the preprocessed user audit data. First, we arrange the discrete log events in chronological order and then structure them into ordered sequences based on their timestamps. The resulting user behavior sequences can effectively capture the evolution and trends of user behavior over time. Through the analysis of these sequences, we can identify regular behavioral patterns as well as potentially anomalous user behaviors. Consequently, the extraction of user behavior sequences from log data represents a crucial step for conducting comprehensive and insightful studies of user sequential information.

In the process of sequence extraction, as illustrated in Fig. 3, user action is classified based on its action type, time of occurrence, and PC environment. The fundamental concept involves assigning a unique identifier to actions that occur within specific time and environmental contexts through a hierarchical framework. Here, the action factor signifies the category of the action, while the PC-time offset encapsulates the encoding associated with a particular time and context. By employing Formula (1), we can compute the action ID for a specific action occurring in a given time and environment. This mechanism allows for the fusion of temporal and contextual information into the action, resulting in the establishment of an action ID.

$$Action\ ID = Action\ Factor \times 8 + PC\text{-}Time\ Offset \tag{1}$$

To comprehensively characterize user behavior, we establish 20 distinct types of action factors and 8 types of PC-time offsets. Notably, the "Logon" and "Logoff" categories in the "Logon" record capture login and logout actions, while the "Connect" and "Disconnect" categories in the "Device" record document device connection and disconnection actions. The "File" category meticulously logs diverse file access actions encompassing compressed files, image files, document files, executable files, text files, and others. The "Email" category records the direction of email communication, including internal to internal (I-I), internal to external (I-O), external to internal (O-I), and external to external (O-O). The "HTTP" category records attributes of visited websites, such as job searching, leaks, cloud storage, hacking, social, and others. Furthermore, the PC-time offset further divides specific user actions based on working time, nonworking time, and the specific type of PC used. This division effectively integrates the user's contextual environment and time into the ensuing sequence. In conclusion, for each user on a given day, a user behavior sequence is formed, denoted as $S_d = [A_{i1}, A_{i2}, ..., A_{ix}]$.

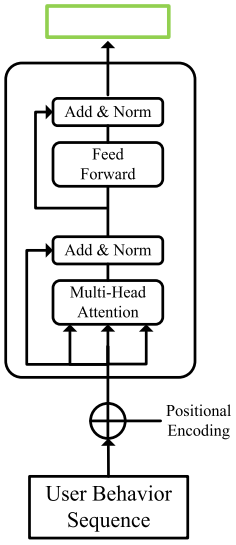


Fig. 4. Sequential analysis model.

3.4.2. Sequential analysis model construction

A neural network architecture called the transformer, which is founded on the attention mechanism introduced by Vaswani et al. (2017), has gained widespread recognition for its remarkable achievements across a spectrum of natural language processing tasks. These tasks encompass machine translation (Wang et al., 2019b), text generation (Hossain et al., 2020), and text classification (Li et al., 2021b). The core concept underlying the transformer is to capture dependencies among different positions within a sequence through a self-attention mechanism, enabling the modeling of global contextual information. This capability empowers the model to construct a representation of comprehensive contextual insights. Comprising an encoder-decoder architecture, the transformer leverages an encoder to encode the input sequence into context-aware representations and a decoder to generate the target sequence using these representations. In the construction of our sequential analysis model, our main focus lies in acquiring contextual representations of sequences to effectively capture their semantic information. To this end, we design our sequential analysis model based on the transformer encoder.

The designed sequential analysis model is illustrated in Fig. 4. In the initial step, the user behavior sequences are input into the positional encoding layer for encoding. The primary function of the positional encoding layer is to explicitly encode the positional information within the input sequence, enabling the model to effectively focus on relevant positions and capture dependencies among elements efficiently. Following this, the encoded output is transmitted to a series of blocks, encompassing multi-head attention, add&norm, and feed-forward mechanisms. These operations are iterated N times, denoting the number of layers contained within the transformer encoder.

3.4.2.1. Positional encoding Positional encoding is utilized to represent the relative positional information within a sequence. It is incorporated by adding positional encoding to the input embedding vector, thereby supplying information regarding the sequence's positional order. This facilitates the model's capture of the inherent order relationships present within the sequence. The positional encoding is computed using the formula as follows:

$$PE_{(pos,2i)} = \sin\left(pos/10000^{2i/d_{model}}\right)$$
$$PE_{(pos,2i+1)} = \cos\left(pos/10000^{2i/d_{model}}\right) \quad (2)$$

The given position index pos and dimension index i are used to compute the positional encoding value. The positional encoding values for each position are calculated using sine and cosine functions, where the position index and the dimension index control the periodicity of these functions. Specifically, both the sine and cosine functions have a period determined by $10000^{2i/d_{model}}$. This approach ensures that each position pos is associated with a unique positional encoding vector.

3.4.2.2. Multi-head attention

By allowing the model to concentrate on crucial segments of the input sequence, the attention mechanism elevates both the performance and expressiveness of the model. Equation (3) elucidates the computation of scaled dot-product attention. This process entails the computation of the dot product score between the query vector Q and the key-value pair (K, V), followed by scaling it to control the gradient magnitude.

$$Attention(Q, K, V) = softmax\left(\frac{QK^T}{\sqrt{d_k}}\right)V \quad (3)$$

Scaled dot-product attention first computes attention weight A by taking the dot product of Q and K, dividing it by a scaling factor, and applying a softmax function. Subsequently, the resulting A are employed to weight the summation of values V, thereby culminating in the ultimate output representation. By utilizing this attention mechanism, the model gains the capability to concentrate on the most pertinent section of the input sequence concerning the query, facilitating the extraction of key information and enhancing the model's expressiveness.

Multi-head attention enhances the model's expressive power by incorporating multiple independent attention heads in parallel. Every attention head operates on a distinct subspace of the input sequence, capturing unique relevance information. By employing separate matrices for queries Q, keys K, and values V, attention weights are computed using scaled dot-product attention, and the corresponding values are then weighted and aggregated. The computational formula for multi-head attention is presented below:

$$MultiHead(Q, K, V) = Concat\left(head_1, \ldots, head_h\right) W^O \quad (4)$$
$$head_i = Attention\left(QW_i^Q, KW_i^K, VW_i^V\right) \quad (5)$$

where W^O, W_i^Q, W_i^K, and W_i^V are parameter matrices. Multi-head attention allows models to compute and capture correlations independently in different subspaces, providing richer representational and expressive capabilities that help improve model performance and generalization.

3.4.2.3. Feed forward

This layer constitutes a fully connected network designed for nonlinear transformation and feature extraction of the hidden state at each position. It encompasses two layers of linear transformation, accompanied by the ReLU activation function. The feed-forward layer's computation formula is shown below:

$$FFN(x) = ReLU\left(xW_1 + b_1\right)W_2 + b_2 \quad (6)$$

where W_1, b_1 is the weight and bias of the first linear transform layer and W_2, b_2 is the weight and bias of the second linear transform layer.

3.4.2.4. Add & norm

The add&norm layer plays a pivotal role within the model by performing residual connections (He et al., 2016) and applying layer normalization operations (Ba et al., 2016). Incorporating a residual connection involves summing the sublayer's output with the input, thereby enabling the information flow from the initial input to subsequent layers. The incorporation of residual concatenation helps address the issues of gradient vanishing and information loss, thereby facilitating the acquisition of meaningful features and representations during the learning process. After that, layer normalization is applied to normalize the combined outputs, reducing variation between dimensions and enhancing model stability and training effectiveness. The incorporation of the add&norm layer effectively accelerates training convergence, improves model generalization, and facilitates the acquisition of valuable features and representations.

3.5. Classification module

In the ultimate classification module, the detection of malicious internal user behaviors is accomplished by leveraging the vectors obtained from both the statistical analysis and sequential analysis modules. The vectors generated by the statistical analysis module $vector_{stat}$ and the sequential analysis module $vector_{sequ}$ are combined using a weighted summation, as shown in Equation (7). The resulting combined vector $vector_{weighted}$ is then fed as input to a fully connected layer, producing a probability distribution over the classes. The class with the highest probability is assigned as the predicted class for the given sample.

$$vector_{weighted} = \alpha \cdot vector_{stat} + (1 - \alpha) \cdot vector_{sequ} \quad (7)$$

To train the network parameters, this study employs the CrossEntropyLoss function as the chosen loss function. The Adam optimizer is utilized for weight updates during the training process.

4. Experimental setup

Within this section, we present a comprehensive overview of the experimental configuration, encompassing details about the experimental environment, dataset, and evaluation framework. First, we present detailed information about the experimental environment, outlining the hardware and software configurations employed for conducting the experiments. Second, we provide an in-depth description of the dataset used in our study, including its source and characteristics. Finally, we elucidate the evaluation framework utilized to assess the performance and effectiveness of our proposed approach.

4.1. Environment

The experiments are conducted with an AMD Ryzen 9 7950X CPU, 64 GB of RAM and an Nvidia GeForce RTX 2080Ti (11 GB) GPU. We conduct our method in the Python 3.9.7 environment. Machine learning-based models are implemented with Scikit-learn (Pedregosa et al., 2011), and deep learning-based models are developed using PyTorch (Paszke et al., 2019).

4.2. Dataset

For the purpose of performing convincing experiments, a publicly accessible CERT insider threat dataset (CERT-IT dataset) r5.2 from Carnegie Mellon University (Lindauer, 2020) is used in this work. The CERT-IT dataset has been extensively utilized in numerous studies on insider threat detection (Wang et al., 2019a; Le et al., 2020; Bartoszewski et al., 2021; Ge et al., 2022). Among its various versions, the r5.2 version offers more comprehensive user behavior information and has been widely employed in previous research. This dataset contains a collection of insider threat scenarios designed to help researchers develop and evaluate insider threat detection algorithms and systems. CERT-IT r5.2 emulates an organization consisting of 2000 employees, including 99 malicious insiders, across four threat scenarios over an 18-month duration. This dataset organizes approximately 38 gigabytes of user behavior logs from several different sources in csv format files.

Table 2
CERT-IT r5.2 dataset description.

Log File	Description
logon.csv	User's log on and off of the system
device.csv	User's device connection information
file.csv	User's file transfer information
email.csv	User's email exchanges
http.csv	User's web browsing history
ldap.csv	Organization structure and user profile
psychometric.csv	User's psychometric scores

These logs encompass a wide array of computer-based operations and user profile information for all employees. The description of these log files is shown in Table 2.

The CERT-IT r5.2 dataset encompasses four distinct scenarios of insider threat behaviors. These insider threat scenarios can be categorized as follows: data exfiltration (Scenario 1), intellectual property theft (Scenarios 2 and 4), and IT sabotage (Scenario 3) (Lindauer, 2020).

4.3. Evaluation framework

In this paper, we employ four performance metrics to assess the effectiveness of the proposed methods and make comparisons. These evaluation metrics encompass the detection rate (DR), precision (PR), F-measure (F1), and false-positive rate (FPR) and are shown in the following formulas. These metrics are widely utilized in the field of classification and are equally applicable to the domain of insider threat detection (Le et al., 2020; Li et al., 2021a). Notably, these metrics rely on counts of true positives (TP), true negatives (TN), false positives (FP), and false negatives (FN) derived from the classification results. In the context of this work, TP refers to malicious insider behaviors that are identified as malicious, while TN refers to benign user behaviors that are classified as benign. FP gives the count of benign user behaviors that are misclassified as malicious, while FN indicates the number of malicious insider behaviors that are misclassified as benign.

Based on the previously derived values, we calculate the performance metrics as follows:

$$DetectionRate = \frac{TP}{TP + FN}$$

The metric known as the detection rate, or recall, quantifies the proportion of accurate positive predictions in relation to all positive instances.

$$Precision = \frac{TP}{TP + FP}$$

Precision denotes the proportion of accurate positive predictions relative to all positive predictions made.

$$F - measure = \frac{2 * TP}{2 * TP + FP + FN}$$

F-measure represents the harmonic mean of precision and recall.

$$FalsePositiveRate = \frac{FP}{FP + TN}$$

The false-positive rate pertains to the proportion of inaccurate positive predictions among all negative instances.

In the CERT-IT r5.2 dataset, benign user behavior accounts for over 99%, whereas malicious insider behavior constitutes less than 0.5%. The presence of such a data imbalance can impact the classification results, as it gives rise to a class imbalance problem. This challenge makes it arduous for models to effectively capture features from classes characterized by limited samples during the training process. Given the substantial variation in the number of instances across different classes, we employ an undersampling technique to restrict the number of benign class items to 20,000. Table 3 shows the experimental dataset statistics, where "Instances" represents the count of user behavior samples and "Activities" represents the number of actions contained in the user behavior samples. The training set is established with a 7:3 ratio relative to the test set.

Table 3
Summary of experimental data statistics.

Class	Instances	Activities
Benign	20,000	2,288,080
Malicious	1,307	175,063

5. Experimental results

In this section, we perform a series of experiments aimed at assessing the effectiveness of our proposed framework. First, we perform experiments for parameter selection, assessing various model parameter settings to determine the optimal configuration. Second, we conduct ablation experiments to investigate the outcomes of removing essential elements from CATE. Third, we perform comparative experiments to assess how CATE performs in comparison to other existing approaches, demonstrating the superiority of CATE. Finally, we validate the effectiveness of CATE in identifying malicious insider threat scenarios.

5.1. Parameter selection

To ensure the optimal performance of our method in subsequent experiments, four key parameters need to be determined: the number of convolution layers in the statistical analysis module, the maximum sequence length, the number of transformer layers in the sequential analysis module, and the number of transformer heads in the sequential analysis module.

During the parameter tuning process, we exclusively utilize the data from the training set to fine-tune the parameters. This approach guarantees the separation of the training data from the test data. To facilitate this, we partition the original training set into two subsets: the parameter-tuning training set and the parameter-tuning validation set. The former is used for training purposes, while the latter is utilized for optimal parameter selection. The split between the parameter-tuning training set and the parameter-tuning validation set follows a 6:1 ratio.

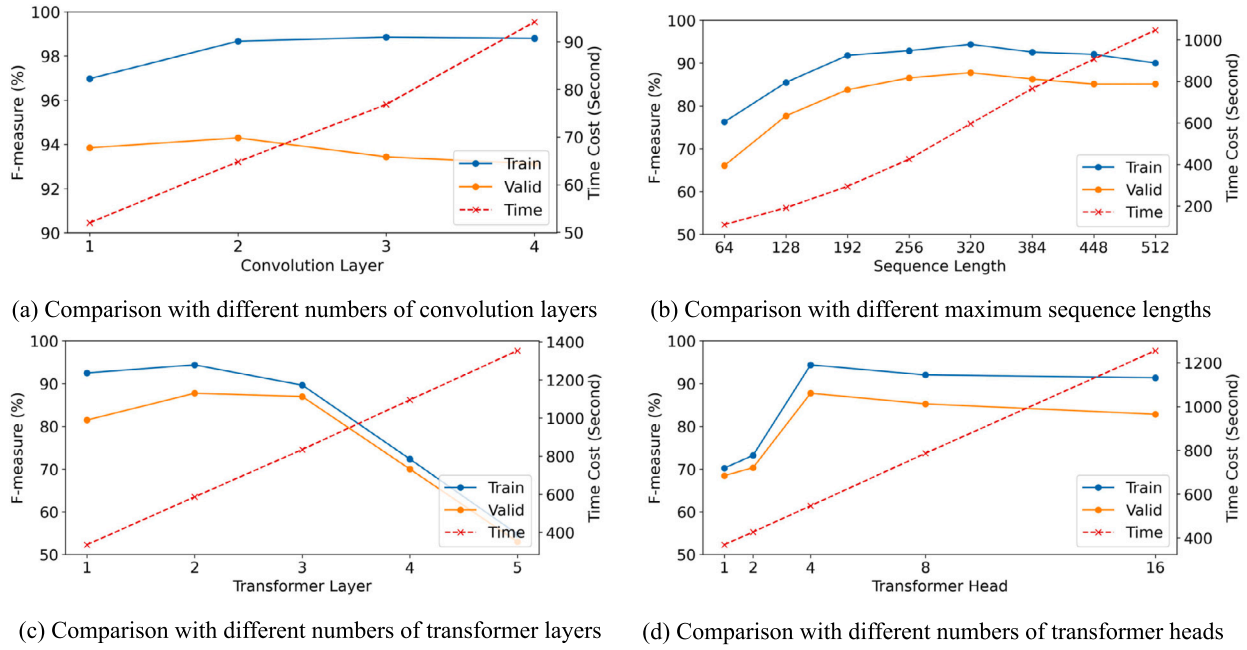
5.1.1. Number of convolution layers

In the statistical analysis module, we adjust the parameters of the convolutional layers within the convolutional network. During the parameter selection process for the number of convolution layers, we conduct experiments across a range of values, spanning from 1 to 4. Intuitively, the inclination might be that augmenting the layers' count would result in enhanced model performance. However, we observe an interesting phenomenon: as the number of layers increases, the training time needed for the model to converge also increases, as depicted in Fig. 5a. More specifically, when the number of convolution layers exceeds 2, both the training and validating performance curves begin to decline, indicating a longer convergence time and potential overfitting of the model.

This unexpected behavior leads us to conclude that excessively deep models are not advantageous for our specific task, as they lead to increased training time and compromised generalization. Hence, based on this observation, we opt to configure the number of convolution layers as 2 in subsequent experiments.

5.1.2. Maximum sequence length

During the parameter selection process for the maximum sequence length, we explore various values to determine the optimal length for input sequences in the sequential analysis module. The length of input sequences is a critical factor in the sequential analysis module, as longer sequences can encompass more contextual information while potentially introducing heightened computational complexity.



(a) Comparison with different numbers of convolution layers

(b) Comparison with different maximum sequence lengths

(c) Comparison with different numbers of transformer layers

(d) Comparison with different numbers of transformer heads

Fig. 5. The figure presents a comparison of F-measure results across various parameter values for the number of convolution layers, maximum sequence length, number of transformer layers, and number of transformer heads. The horizontal axis of the figure indicates the different values of each parameter, while the vertical axis represents the corresponding F-measure values for each parameter configuration. The training F-measure fitting curve is indicated by the solid blue line, the validating F-measure fitting curve is shown by the solid orange line, and the dash-dotted red x line illustrates the fitting curve for time cost. (For interpretation of the colors in the figure(s), the reader is referred to the web version of this article.)

As shown in Fig. 5b, the sequence length of 320 demonstrates superior performance compared to the other examined lengths. This particular length strikes a favorable balance, providing sufficient contextual information while maintaining relatively high computational efficiency. Compared to shorter sequences, a sequence length of 320 more effectively captures intricate patterns and long-range dependencies inherent in the data, thereby enhancing model performance. On the other hand, longer sequence lengths (such as 384, 448, and 512) offer the advantage of deeper contextual comprehension. However, they also introduce significant computational burdens, resulting in longer training times. Conversely, shorter sequence lengths (e.g., 64, 128, 192, and 256) have some advantages in computational efficiency. They might limit the model's ability to comprehend the global context, thereby affecting overall performance. Hence, the maximum sequence length is set to 320 in subsequent experiments.

5.1.3. Number of transformer layers

As shown in Fig. 5c, we explore a range of values from 1 to 5 for the number of transformer layers for parameter tuning. The objective is to pinpoint the most suitable number of transformer layers that would yield optimal performance for our specific task.

Upon analyzing the experimental results, we observe an interesting trend. While increasing the number of transformer layers beyond two initially leads to improvements in performance, further increments result in diminishing returns. Specifically, the model's performance plateaus and even exhibits a decline when using more than two transformer layers.

This phenomenon suggests that a deeper model with more than two transformer layers could suffer from overfitting or struggle to effectively capture the relevant information pertinent to the task. Thus, we choose a compromise for the number of transformer layers by taking the value 2.

5.1.4. Number of transformer heads

As depicted in Fig. 5d, we conduct parameter tuning for the number of transformer heads, exploring five distinct values. The experimental results reveal an intriguing pattern. When using one or two transformer heads, the model's performance shows limited improvement, suggesting that these configurations struggle to adequately capture the intricate patterns and dependencies within the data. However, as the number of transformer heads increases to four, a significant performance boost is observed. Enhanced is the model's capacity to process input data, attend to various aspects, and create more precise and meaningful data representations.

Further experiments with more than four transformer heads do not yield significant performance improvements. This implies that augmenting the number of heads does not always result in improved results and introduce unnecessary computational overhead. On balance, we choose four as the number of transformer heads.

5.2. Ablation study

In the ablation study, we conduct a detailed investigation of the statistical analysis module (referred to as CA, which includes the convolutional layers and attention layers) and the sequential analysis module (referred to as TE, which represents the transformer encoder). Additionally, we compare the performance of our method when combining both modules, referred to as CATE. To assess the contributions of CA and TE, we perform a series of ablation experiments by individually removing these modules and comparing them to the complete CATE method. Specifically, we construct three ablation models: 1) a model containing only the statistical analysis module; 2) a model containing only the sequential analysis module; and 3) the complete CATE model.

Fig. 6 illustrates the metrics used to compare the results. CATE outperforms CA and TE across all metrics, providing evidence for the effectiveness of the CATE framework design. Removing the CA module led to a significant decrease in model performance. The CA module enables the model to capture critical information and features from the statistical data, thereby enhancing performance. Similarly, the TE module showed significant importance. Responsible for sequence-level modeling and understanding through attention mechanisms on input sequences, it helped capture sequential patterns of user behavior, leading

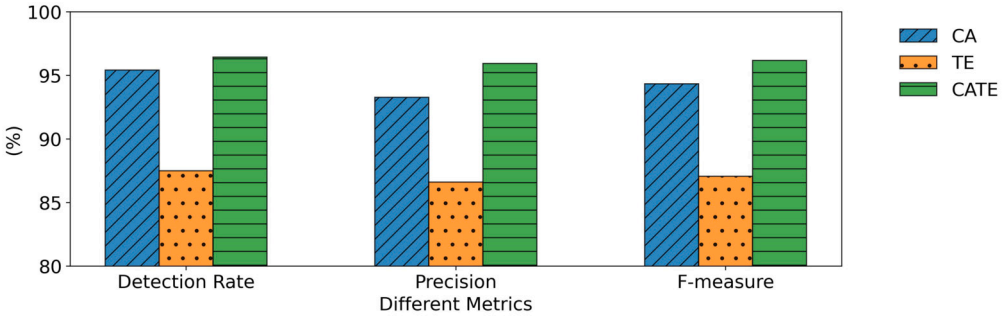


Fig. 6. This figure shows the results of our ablation study, which aimed to assess performance impacts by selectively removing different modules.

Table 4
Comparative results (%).

Method	Detection Rate	Precision	F-measure	False Positive Rate
LR	68.62	85.66	76.20	0.74
KNN	88.26	86.28	87.26	0.91
DT	91.83	85.91	88.77	0.98
RF	92.34	93.29	92.82	0.43
GB	86.22	91.59	88.83	0.51
M-LR	69.38	80.95	74.72	1.06
M-MLP	79.84	86.46	83.02	0.81
M-RF	78.31	95.63	86.11	0.23
M-XGB	91.58	93.01	92.28	0.44
CNN	93.87	92.92	93.40	0.46
LSTM	85.96	81.99	83.93	1.23
Transformer	87.50	86.62	87.06	0.88
CNN+LSTM	94.64	93.92	94.28	0.39
CATE	96.42	95.93	96.18	0.26

to performance improvements. The absence of the TE module affects the model's performance. Further comparison of the three ablation models reveals that the CATE method, which combines CA and TE, exhibits the best performance. The CATE method effectively leverages the advantages of both CA and TE modules, achieving comprehensive data analysis and modeling, resulting in optimal performance.

The ablation study elucidates the critical roles of the statistical analysis module (CA) and the sequential analysis module (TE) in our approach. They are responsible for user statistical and sequential information learning, respectively, and both play important roles in our tasks. The fusion of CA and TE in the CATE method demonstrates a synergistic effect of the two modules, resulting in superior performance.

5.3. Comparison study

We compare our proposed CATE method against state-of-the-art techniques, including traditional machine learning and deep learning methods. Table 4 summarizes the results of these comparative experiments. To ensure a fair and robust comparison, we reproduce these methods and utilize the same dataset and random seeds as CATE during the training and testing phases. We explore the parameter space within each method, obtaining optimal results for all methods under the same experimental condition.

First, we compare our CATE method with traditional machine learning techniques, including logistic regression (LR), k-nearest neighbor (KNN), decision tree (DT), random forest (RF), and gradient boosting (GB). The above models are fed with features consistent with those extracted in the statistical analysis module.

We have also compared the four machine learning methods proposed by Le et al. (2020), and we have first replicated the feature extraction process of Le et al. (2020) and conducted experiments on the dataset of this paper using the models mentioned in the Le et al. (2020). They manually crafted a set of 824 features for insider threat scenarios and modeled their manually constructed features using four different machine learning models: logistic regression (M-LR), multilayer perceptron (M-MLP), random forest (M-RF), and XGBoost (M-XGB).

Across all metrics, CATE consistently exhibits significantly superior performance compared to traditional machine learning-based methods, as evidenced by the data presented in the initial nine rows of Table 4. Notably, the M-RF method has a lower false alarm rate, the detection rate of this method is very low at the same time, and the method produces more missed alarms and is not of practical value. The neural network utilized in our proposed CATE method is the key factor behind the superiority of deep learning methods over traditional machine learning techniques. The neural network is capable of learning more abstract and advanced feature representations, enabling the capture of intricate patterns and relationships within the user data.

Regarding deep learning-based techniques, we evaluate our approach alongside methods such as the convolutional neural network (CNN), long short-term memory (LSTM), and transformer models. The CNN, LSTM, and transformer models are all commonly used in anomaly detection tasks Tekerek and Yapici (2022); Aydın et al. (2022); Huang et al. (2021). Simultaneously, we also combine CNN and LSTM models (CNN+LSTM) to learn both statistical and sequential information about the user.

In the CNN approach, we employ features extracted from the statistical analysis module, applying equivalent feature transformations as our proposed method. Subsequently, these features are input into the CNN model for learning. For the LSTM or transformer models, we utilize sequences from the sequential analysis module as inputs for the respective LSTM or transformer models. Moreover, we combine CNN and LSTM models, enabling the CNN model to learn features from the statistical analysis module and the LSTM model to comprehend sequences from the sequential analysis module. This integration enables the simultaneous acquisition of the user's statistical and sequential information.

The experimental results depicted in Rows 10 to 13 show that the CATE method still outperforms the compared deep learning methods. Despite the CNN+LSTM method's ability to simultaneously capture the

Table 5
Comparative results of malicious scenario identification (%).

Method	Detection Rate	Precision	F-measure	False Positive Rate	DE-IPT-IS
LR	84.34	94.51	88.69	7.52	17-253-6
KNN	85.42	96.21	88.90	3.16	13-327-6
DT	92.77	92.88	92.82	3.19	21-317-6
RF	91.57	97.86	94.42	3.49	20-313-6
GB	92.25	94.30	93.25	3.72	21-309-6
M-LR	83.89	94.55	88.76	7.65	21-250-5
M-MLP	82.61	94.95	87.84	4.99	21-291-4
M-RF	86.02	98.36	91.59	5.67	21-278-5
M-XGB	92.55	96.47	94.33	3.45	21-314-6
CNN	93.35	96.15	94.66	2.64	21-325-6
LSTM	85.81	87.61	86.69	5.77	21-278-5
Transformer	92.58	95.10	93.64	3.43	21-316-6
CNN+LSTM	93.88	96.91	95.31	2.15	21-332-6
CATE	95.97	97.95	96.85	1.14	22-348-6

user's statistical and sequential information, which results in improved performance in contrast to individual CNN or LSTM models, it still falls short of matching the performance achieved by the CATE method. These results illustrate the effectiveness and superiority of the CATE method we have proposed.

By leveraging the strengths of both the statistical analysis module and the sequential analysis module, the CATE method can understand and process user data more comprehensively, resulting in superior performance in insider threat detection tasks. Our CATE method achieves an F-measure of 96.18% while maintaining a false-positive rate of just 0.26%. The outcomes from comparative experiments further validate the advantages of the CATE method in combining statistical and sequential information.

5.4. Malicious scenario identification

In this section, we investigate the capability of our method for identifying specific insider threat scenarios. As previously outlined in Section 4.2, these scenarios can be classified into three categories: data exfiltration (DE), intellectual property theft (IPT), and IT sabotage (IS). We retrain our method, as well as the state-of-the-art methods described in Section 5.3, to adapt to the multi-class classification task and conduct testing on the same dataset.

Table 5 presents the comparative results of our proposed CATE method against state-of-the-art methods in malicious scenario identification. The first column of Table 5 lists the detection methods, and the second to fifth columns detail the detection metrics. The final column shows the actual detection values for identifying malicious scenarios. The comparative results highlight our CATE method's superiority over existing methods. In the IPT scenario category, the actual detection values of various methods reveal the shortcomings of current detection techniques. The traditional machine learning method, KNN, successfully detects 327 instances in IPT scenarios. However, its effectiveness markedly declines in other malicious scenarios. Moreover, M-RF methods show higher precision, but they face challenges in accurately identifying IPT scenarios. Existing deep learning methods reach a maximum of 332 IPT detections, but they still fall short of CATE's performance. CATE's effectiveness further illustrates that incorporating both statistical and sequential information enhances the identification of malicious scenarios.

Subsequently, we conduct an in-depth analysis of the limitations of the CATE method in identifying malicious scenarios, focusing on the root causes of these limitations. The confusion matrix depicted in Fig. 7 shows the CATE method's detection results in malicious scenario identification. The confusion matrix result illustrates CATE's commendable performance in identifying DE and IS scenarios. However, it demonstrates relatively suboptimal results in identifying the IPT scenario. Hence, our investigation concentrates on comprehensively understanding the limitations of CATE, specifically within the IPT scenario. Our detailed examination of correctly and incorrectly predicted IPT scenario instances revealed that, in correctly predicted cases, the average proportion of malicious to total daily activities is 10.14%. In contrast, this ratio significantly drops to 3.19% in incorrectly predicted instances. These findings indicate that the CATE method is more effective at identifying instances where malicious activities represent a higher proportion of total daily activities. Nevertheless, its detection performance decreases when malicious activities represent a lower proportion of total daily activities. Future research should focus on investigating the sparsity of malicious activities and aim to develop methodologies targeted at detecting low-frequency malicious activities.

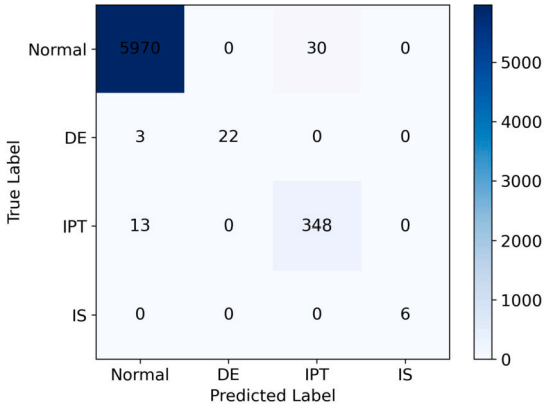


Fig. 7. This figure shows the confusion matrix result of our method for malicious scenario identification.

We also conduct an analysis of the duration of three distinct malicious scenarios, as depicted in Fig. 8. The median duration of the DE scenario is 5.99 days, while the IS scenario has a notably shorter median of 1.41 days. In contrast, the IPT scenario demonstrates a substantially longer median duration of 58.31 days, with the longest case lasting up to 160 days. Moreover, within the IPT scenario, malicious activities occur more sporadically than in the DE and IS scenarios, characterized by their dispersed or intermittent execution. This indicates that, in comparison to other scenarios, malicious activities in IPT are more intricately intertwined with users' regular activities, demonstrating heightened concealment and thereby posing greater challenges for detection. The high level of concealment significantly hampers the detection performance of the IPT scenario. In contrast to the DE and IS scenarios, where malicious patterns appear more frequently and consistently, the infrequency of malicious activities in the IPT scenario results in a lack of regularity, complicating detection and impeding effective detection of such behaviors.

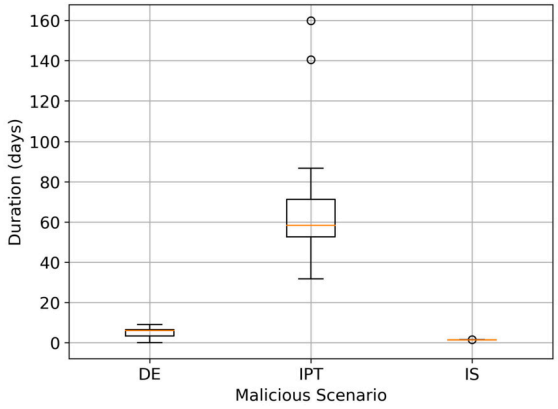


Fig. 8. This figure shows the duration analysis of distinct malicious scenarios.

Our proposed CATE method exhibits enhanced capabilities for identifying malicious scenarios. Moreover, in the identification of the IPT scenario, the CATE method also outperforms existing approaches. However, due to the low-frequency operational patterns, prolonged duration, and high level of concealment inherent in IPT scenarios, focusing solely on daily granularity is insufficient for effective detection. Future research can explore integrating daily contextual information, such as analyzing pre-detection user log data alongside historical and current-day records to highlight access frequency deviations. Additionally, the multi-granularity fusion approach for detecting insider threats deserves attention. By merging data across multiple granularities, the method consolidates user behaviors over various timeframes, enhancing the identification of low-frequency malicious activity.

6. Conclusion

In this paper, we propose a comprehensive framework for insider threat detection based on statistical and sequential analysis and implement it utilizing the Convolutional Attention and Transformer Encoder (CATE). Our framework leverages the strengths of both statistical analysis and sequential analysis modules to comprehensively analyze and model the input user data. The statistical analysis module effectively extracts valuable features from the data, while the sequential analysis module captures temporal dependencies and patterns. By combining these modules, our framework demonstrates robust performance and achieves state-of-the-art results in detecting insider threats.

There are several aspects that we can continue to explore in future work. First, we can introduce more advanced transformer architectures into the model's structure to enhance its capability in understanding user temporal behaviors. Second, the pursuit of techniques to refine the identification of intricate insider threat scenarios, such as intellectual property theft, remains imperative. The identification of these complex scenarios is pivotal in fortifying the protection of sensitive information and mitigating insider threats. Based on the analysis of the malicious scenario identification section, future research can focus on the low-frequency operational patterns, the prolonged durations, and the high level of concealment that characterize intellectual property theft scenarios. To achieve this advancement, incorporate day-level contextual information or explore multi-granularity fusion approaches, therefore increasing the capability of methods for identifying complex insider threats. Third, our work requires more interpretability. In practical applications, it is crucial to trace and identify which specific actions lead to the occurrence of malicious behaviors. Thus, incorporating more interpretability techniques into our method to provide clearer explanations of the model's decision-making process is an important avenue for improvement. Finally, the framework of combining statistical and sequential analysis can be extended to other domains beyond insider threat detection. For instance, applying it to other security domains, financial risk prediction, and more will contribute to enhancing the overall performance of data analysis and modeling.

CRediT authorship contribution statement

Haitao Xiao: Conceptualization, Investigation, Methodology, Software, Writing – original draft, Writing – review & editing. **Yan Zhu:** Software, Validation. **Bin Zhang:** Software, Validation. **Zhigang Lu:** Resources, Supervision. **Dan Du:** Formal analysis, Writing – review & editing. **Yuling Liu:** Project administration, Resources, Writing – review & editing.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Data will be made available on request.

Acknowledgements

The financial support for this study is provided by the Strategic Priority Research Program of the Chinese Academy of Sciences [Grant Number XDC02040100], the Science and Technology Project of the State Grid Corporation of China [Grant Number 5700-202352606A-3-2-ZN], and the Youth Innovation Promotion Association of the Chinese Academy of Sciences [Grant Number 2021156]. Additionally, this study is also supported by the Program of the Key Laboratory of Network Assessment Technology at the Chinese Academy of Sciences and the Program of the Beijing Key Laboratory of Network Security and Protection Technology.

References

Afzal, M.Z., Capobianco, S., Malik, M.I., Marinai, S., Breuel, T.M., Dengel, A., Liwicki, M., 2015. Deepdocclassifier: document classification with deep convolutional neural network. In: 2015 13th International Conference on Document Analysis and Recognition (ICDAR). IEEE, pp. 1111–1115.

Aydın, H., Orman, Z., Aydın, M.A., 2022. A long short-term memory (lstm)-based distributed denial of service (ddos) detection and defense system design in public cloud network environment. Comput. Secur. 118, 102725.

Ba, J.L., Kiros, J.R., Hinton, G.E., 2016. Layer normalization. arXiv preprint. arXiv:1607.06450.

Bartoszewski, F.W., Just, M., Lones, M.A., Mandrychenko, O., 2021. Anomaly detection for insider threats: an objective comparison of machine learning models and ensembles. In: ICT Systems Security and Privacy Protection: 36th IFIP TC 11 International Conference. SEC 2021, Oslo, Norway, June 22–24, 2021. In: Proceedings. Springer, pp. 367–381.

Cullen, M., Russell, S., Bosshardt, M., Juraska, S., Stellmack, A., Duehr, E., Jeansonne, K., 2011. Five-factor model of personality and counterproductive cyber behaviors. In: Poster Presented at the Annual Conference of the Society for Industrial and Organizational Psychology. Chicago, IL.

Du, M., Li, F., Zheng, G., Srikumar, V., 2017. Deeplog: anomaly detection and diagnosis from system logs through deep learning. In: Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security, pp. 1285–1298.

Gayathri, R., Sajjanhar, A., Xiang, Y., 2020. Image-based feature representation for insider threat classification. Appl. Sci. 10, 4945.

Ge, D., Zhong, S., Chen, K., 2022. Multi-source data fusion for insider threat detection using residual networks. In: 2022 3rd International Conference on Electronics, Communications and Information Technology (CECIT). IEEE, pp. 359–366.

Gu, J., Wang, Z., Kuen, J., Ma, L., Shahroudy, A., Shuai, B., Liu, T., Wang, X., Wang, G., Cai, J., et al., 2018. Recent advances in convolutional neural networks. Pattern Recognit. 77, 354–377.

Gurucul, 2023. 2023 Insider Threat Report. Technical Report. Gurucul.

He, K., Zhang, X., Ren, S., Sun, J., 2016. Deep residual learning for image recognition. In: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, pp. 770–778.

He, W., Wu, X., Wu, J., Xie, X., Qiu, L., Sun, L., 2021. Insider threat detection based on user historical behavior and attention mechanism. In: 2021 IEEE Sixth International Conference on Data Science in Cyberspace (DSC). IEEE, pp. 564–569.

Hossain, N., Ghazvininejad, M., Zettlemoyer, L., 2020. Simple and effective retrieve-edit-rerank text generation. In: Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics, pp. 2532–2538.

Huang, W., Zhu, H., Li, C., Lv, Q., Wang, Y., Yang, H., 2021. Itdbert: temporal-semantic representation for insider threat detection. In: 2021 IEEE Symposium on Computers and Communications (ISCC). IEEE, pp. 1–7.

Le, D.C., Zincir-Heywood, N., Heywood, M.I., 2020. Analyzing data granularity levels for insider threat detection using machine learning. IEEE Trans. Netw. Serv. Manag. 17, 30–44.

Li, D., Yang, L., Zhang, H., Wang, X., Ma, L., Xiao, J., 2021a. Image-based insider threat detection via geometric transformation. Secur. Commun. Netw. 2021, 1–18.

Li, P., Zhong, P., Mao, K., Wang, D., Yang, X., Liu, Y., Yin, J., See, S., 2021b. Act: an attentive convolutional transformer for efficient text classification. In: Proceedings of the AAAI Conference on Artificial Intelligence, pp. 13261–13269.

Li, Z., Liu, K., 2020. An event based detection of internal threat to information system. In: Advances in Harmony Search, Soft Computing and Applications, vol. 15. Springer, pp. 44–53.

Lindauer, B., 2020. Insider threat test dataset. https://kilthub.cmu.edu/articles/dataset/Insider_Threat_Test_Dataset/12841247. 10.1184/R1/12841247.v1.

Liu, L., De Vel, O., Chen, C., Zhang, J., Xiang, Y., 2018. Anomaly-based insider threat detection using deep autoencoders. In: 2018 IEEE International Conference on Data Mining Workshops (ICDMW). IEEE, pp. 39–48.

Liu, Q., Zhang, N., Yang, W., Wang, S., Cui, Z., Chen, X., Chen, L., 2017. A review of image recognition with deep convolutional neural network. In: Intelligent Computing Theories and Application: 13th International Conference. ICIC 2017, Liverpool, UK, August 7–10, 2017. In: Proceedings, Part I, vol. 13. Springer, pp. 69–80.

Nguyen, N., Reiher, P., Kuenning, G.H., 2003. Detecting insider threats by monitoring system call activity. In: IEEE Systems, Man and Cybernetics Society Information Assurance Workshop, 2003. IEEE, pp. 45–52.

Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z., Gimelshein, N., Antiga, L., et al., 2019. Pytorch: an imperative style, high-performance deep learning library. Adv. Neural Inf. Process. Syst. 32.

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., et al., 2011. Scikit-learn: machine learning in python. J. Mach. Learn. Res. 12, 2825–2830.

Ponemon, 2022. 2022 Cost of Insider Threats Global Report. Technical Report. Ponemon Institute.

Sun, Y., Xue, B., Zhang, M., Yen, G.G., Lv, J., 2020. Automatically designing cnn architectures using the genetic algorithm for image classification. IEEE Trans. Cybern. 50, 3840–3854.

Tekerek, A., Yapici, M.M., 2022. A novel malware classification and augmentation model based on convolutional neural network. Comput. Secur. 112, 102515.

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., Kaiser, Ł., Polosukhin, I., 2017. Attention is all you need. Adv. Neural Inf. Process. Syst. 30.

Wang, J., Cai, L., Yu, A., Meng, D., 2019a. Embedding learning with heterogeneous event sequence for insider threat detection. In: 2019 IEEE 31st International Conference on Tools with Artificial Intelligence (ICTAI). IEEE, pp. 947–954.

Wang, Q., Li, B., Xiao, T., Zhu, J., Li, C., Wong, D.F., Chao, L.S., 2019b. Learning deep transformer models for machine translation. In: Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics, pp. 1810–1822.

Xie, X., Cheng, G., Wang, J., Yao, X., Han, J., 2021. Oriented r-cnn for object detection. In: Proceedings of the IEEE/CVF International Conference on Computer Vision, pp. 3520–3529.

Yuan, F., Cao, Y., Shang, Y., Liu, Y., Tan, J., Fang, B., 2018. Insider threat detection with deep neural network. In: Computational Science–ICCS 2018: 18th International Conference. Wuxi, China, June 11–13, 2018. In: Proceedings, Part I, vol. 18. Springer, pp. 43–54.

Yuan, S., Wu, X., 2021. Deep learning for insider threat detection: review, challenges and opportunities. Comput. Secur. 104, 102221.

Zhang, Z., Wang, S., Lu, G., 2020. An internal threat detection model based on denoising autoencoders. In: Advances in Intelligent Information Hiding and Multimedia Signal Processing: Proceedings of the 15th International Conference on IIH-MSP in Conjunction with the 12th International Conference on FITAT. July 18–20, Jilin, China, vol. 2. Springer, pp. 391–400.

Zhu, D., Huang, X., Li, N., Sun, H., Liu, M., Liu, J., 2022. Rap-net: a resource access pattern network for insider threat detection. In: 2022 International Joint Conference on Neural Networks (IJCNN). IEEE, pp. 1–8.

Haitao Xiao received the B.S. degree from Hubei University in 2019. He is currently pursuing the Ph.D. degree at the Institute of Information Engineering, Chinese Academy of Sciences. His current research interests include data mining and cybersecurity.

Yan Zhu received the M.S. degree from Capital Normal University in 2019. She is currently working at the Institute of Information Engineering, Chinese Academy of Sciences. Her research interests include network security situational awareness, security measurement, and visualization analysis.

Bin Zhang received the Ph.D. degree from Beijing University of Posts and Telecommunications. He is currently a senior engineer at the China Cybersecurity Review Technology and Certification Center. His research interests include network security measurement.

Zhigang Lu received the Ph.D. degree from the Chinese Academy of Sciences in 2010. He is a professor at the Institute of Information Engineering, Chinese Academy of Sciences. His research focuses on network and system security, specifically cyber situation awareness, intrusion detection and prevention, as well as mobile terminal security.

Dan Du received the M.S. degree from the Institute of Information Engineering, Chinese Academy of Sciences in 2016. She is currently an engineer at the Institute of Information Engineering, Chinese Academy of Sciences. Her research interests include network security situational awareness, security measurement, and mobile terminal security.

Yuling Liu received his Ph.D. degree from the Institute of Software, Chinese Academy of Sciences in China. He is currently a senior engineer at the Institute of Information Engineering, Chinese Academy of Sciences. His research areas are network security situational awareness, network security, big data analysis, and security measurement and certification.